LP-Pruned Quantum Branch-and-Bound for the 0/1 Knapsack Problem: Query Analysis and Spectral Checks

Nagini S.[1], Madhavi A.[1*], Shiva Sai[1], Metta Kumudini[1], Purusharth Mishra[1], Rakshit Chaturvedi[1], Veerabhadra Yerram[1]

[1]Department of Computer Science and Engineering, VNR VJIET, Hyderabad, India.

*Corresponding author(s). E-mail(s): madhavi_a@vnrvjiet.in;
Contributing authors: nagini_s@vnrvjiet.in;
venkatashivasai07@gmail.com; kumudini.metta@gmail.com;
purusharthmishra10@gmail.com; rakshitchaturvedi09@gmail.com;
veerabhadrayerram@gmail.com;

Abstract

We study a hybrid algorithm for integer knapsack combining exact linear programming variable fixing with quantum tree detection. We specify witness recovery and account for accumulated decision error under an explicit tree-size bound. A core containment result and an integer counterexample describe the role of efficiency spacing. We derive a safe fixed-point pruning rule and synthesize reversible circuits for it. Classical benchmarks, sparse statevector circuit checks, and simulations of explicit walk operators test distinct components. These results establish neither a complete quantum walk implementation nor a practical quantum speedup.

Keywords: quantum computing, knapsack problem, branch-and-bound, quantum walk, linear programming, variable fixing

1 Introduction

The 0/1 knapsack problem is one of the most studied problems in combinatorial optimization: given n items, each with a weight w_i and a value v_i, and a capacity W,

find the subset that maximizes total value without exceeding the capacity. Despite its simple formulation, the problem is NP-hard, and the classical Horowitz–Sahni meet-in-the-middle approach runs in $\widetilde{O}(2^{n/2})$ time and $O(2^{n/2})$ space [1].

Quantum computing has long promised speedups for combinatorial problems. Grover's algorithm [2] provides a quadratic speedup for unstructured search, and its generalization via amplitude amplification [3] is a powerful tool. However, applying Grover search directly to knapsack uses $O(2^{n/2})$ threshold-oracle queries; this alone does not improve the classical meet-in-the-middle exponential rate.

Quantum backtracking and branch-and-bound provide oracle-model query bounds for structured search trees [4, 5]. Their applicability depends on constructing valid branching and bounding predicates and accounting for both detection and solution recovery. We study how exact knapsack LP fixing and finite-precision node bounds fit this framework.

1.1 Contributions and Novelty

Our algorithm combines exact LP reduced-cost fixing with quantum tree detection. The contributions are:

1. A core containment bound and an integer family showing that the LP gap alone cannot bound the number of unfixed items.
2. A threshold-search procedure with explicit witness recovery and a query bound conditional on a certified bound for every queried tree.
3. A finite-precision pruning lemma and reversible node-predicate circuits with clean workspace.
4. Component checks on specified circuits and seeds, including the 700-instance classical benchmark and the completed subset of a spectral audit.
5. Classical comparisons with OR-Tools, including a separate large-coefficient benchmark with an independent exact reference.

We do not claim priority for hybrid quantum knapsack search, wall-clock quantum acceleration, or a complete gate-complexity bound for the full walk. The synthesized node predicates and the simulated walk operators are separate components.

The remainder of this paper reviews related work, defines the algorithm and its assumptions, gives the analysis, and reports the component experiments and limitations.

2 Literature Review

Classical exact methods. Dantzig's fractional relaxation [6], meet-in-the-middle [1], dynamic programming, and core methods [7, 8] provide the classical context. Pisinger [8] studies core difficulty and credits the earlier core concept to Balas and Zemel. We use exact forced-item LP tests; their validity follows from the relaxation upper bound, not from an assumed small core. Specialized solvers such as COMBO combine dynamic programming and strong bounds [9].

Quantum tree search. Montanaro's backtracking results distinguish detection (Theorem 1.1) from finding an assignment (Theorem 1.2) [4]. His branch-and-bound

Table 1 Technical comparison with closely related work. These are differences in method, not claims that the present work dominates prior algorithms.

Method	Search mechanism and scope
Montanaro [5]	Generic quantum branch-and-bound with tree-size estimation and witness search.
Wilkening et al. [10]	Feasible-solution superpositions, amplitude amplification, and maximum finding.
Creemers–Pérez Armas [11]	Grover-based binary search and hybrid branch-and-bound.
Bożejko et al. [12]	Annealer-assisted classical branch-and-bound.
Present work	Exact LP fixing, a conditional detection/recovery composition, and reversible pruning components.

algorithm additionally controls tree size while searching for an optimal solution [5]. Our binary-search and prefix-recovery procedure uses the detection result with a supplied tree-size bound; it does not implement that adaptive tree-size-estimation algorithm. Grover search and amplitude amplification supply alternative search primitives [2, 3].

Closest quantum knapsack work. Wilkening et al. [10] combine a Quantum Tree Generator with amplitude amplification and quantum maximum finding, and give resource estimates and classical-solver comparisons. Their method has theoretical guarantees; it is not restricted to a variational QAOA routine. Creemers and Pérez Armas [11] develop quantum binary search and hybrid branch-and-bound for knapsack and other discrete optimization problems. Bożejko et al. [12] study a hybrid branch-and-bound method using a D-Wave annealer. Consequently, hybrid search, thresholding, and knapsack specificity are not by themselves new contributions here. The present focus is the exact fixing/core analysis, safe finite-precision node predicates, and explicit spectral checks under stated oracle-model assumptions.

3 Preliminaries

3.1 Problem Definition

The 0/1 Knapsack Problem is defined as follows. Given a set of n items, where item i has weight $w_i \in \mathbb{Z}^+$ and value $v_i \in \mathbb{Z}^+$, and a capacity $W \in \mathbb{Z}^+$, find a binary vector $x^* = (x_1^*, x_2^*, \ldots, x_n^*) \in \{0,1\}^n$ that solves

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{subject to} \quad \sum_{i=1}^{n} w_i x_i \leq W. \tag{1}$$

We denote the optimal value by $Z^* = \sum_i v_i x_i^*$. Without loss of generality, we assume $w_i \leq W$ for all i (items heavier than the capacity are trivially excluded). Efficiencies $e_i = v_i/w_i$ may tie; ties are broken deterministically using the original item order. Distinctness is required only for the spacing bound below. If all items fit, the instance is solved directly.

3.2 LP Relaxation, Split Item, and Dantzig Bound

The LP relaxation of (1) permits fractional assignments $x_i \in [0, 1]$. Its solution has a closed-form structure due to Dantzig [6].

Definition 1 (Efficiency Ordering). Order items so that $e_1 \geq e_2 \geq \cdots \geq e_n$ where $e_i = v_i/w_i$.

Definition 2 (Split Item). The *split item* s is the smallest index such that

$$\sum_{i=1}^{s} w_i > W \quad \text{and} \quad \sum_{i=1}^{s-1} w_i \leq W. \tag{2}$$

The LP optimal solution is then

$$x_i^{LP} = \begin{cases} 1 & \text{if } i < s, \\ (W - \sum_{j=1}^{s-1} w_j)/w_s & \text{if } i = s, \\ 0 & \text{if } i > s, \end{cases} \tag{3}$$

and the *Dantzig bound* is $Z_{LP} = \sum_{i=1}^{n} v_i x_i^{LP}$. This is computable in $O(n)$ time after sorting, and satisfies $Z^* \leq Z_{LP}$.

Definition 3 (Core Items). Given a parameter $\delta \geq 0$, the *core* C is the set of items with efficiencies close to the split item:

$$C = \{i : |e_i - e_s| \leq \delta \cdot e_s\}. \tag{4}$$

This efficiency band is an analytical comparison set, not a fixing rule. The actual partition (F_1, C, F_0) is produced by Algorithm 1. The core literature [8] motivates studying the items near the split; it does not guarantee a small core for every instance.

3.3 Amplitude Amplification

We use the following standard result [3].

Brassard et al. [3], Theorem 3, give expected $O(1/\sqrt{p})$ uses of a measurement-free preparation algorithm and its inverse to find a good state when its initial success probability is $p > 0$. The good-state predicate must be supplied; this statement alone does not give an optimal knapsack witness.

3.4 Quantum Tree Detection (Montanaro)

Theorem (Montanaro [4], Theorem 1.1). Given an upper bound B on the number of vertices of a backtracking tree on m Boolean variables, and coherent access to its predicate and deterministic branching heuristic, existence of a marked vertex can be detected with failure probability at most η using

$$O\Big(\sqrt{B \max(1, m)} \log(1/\eta)\Big) \tag{5}$$

queries. Here the predicate rejects infeasible prefixes or prefixes with LP bound below the fixed threshold, accepts complete feasible assignments meeting the threshold, and is indeterminate otherwise. The branching heuristic selects the next item in the fixed efficiency order. Montanaro's branching heuristic, also named h in that source, is distinct from our LP bounding predicate h.

Detection does not return an assignment. Theorem 1.2 of the same source gives a separate witness-search result; our explicit recovery procedure below instead composes further detections. The bound is in oracle queries, excludes the cost of certifying B, and does not establish a gate count or wall-clock improvement.

3.5 Reduced-Cost Variable Fixing

Given a lower bound Z_{LB} (e.g., from a greedy solution), an item i with $x_i^{LP} = 1$ can be *provably fixed* to 1 if removing it causes the LP upper bound to drop below Z_{LB}. Similarly, an item j with $x_j^{LP} = 0$ can be fixed to 0 if forcing it in causes the LP bound to drop below Z_{LB}. These are exact relaxation-based tests: the opposite assignment cannot belong to any optimum if its upper bound is below a feasible lower bound.

3.6 Meet-in-the-Middle Framework

The Horowitz–Sahni algorithm [1] partitions the n items into two disjoint sets A and B with $|A| = |B| = n/2$. It enumerates all $2^{n/2}$ subsets of A, storing pairs $(w(S_A), v(S_A))$ for each subset $S_A \subseteq A$, where $w(S_A) = \sum_{i \in S_A} w_i$ and $v(S_A) = \sum_{i \in S_A} v_i$. It does the same for B. Then, for each subset S_A of A, it finds the best compatible subset S_B of B satisfying $w(S_A)+w(S_B) \leq W$ using binary search on the sorted list of B-subsets. This straightforward implementation uses $O(n2^{\lceil n/2 \rceil})$ arithmetic operations and $O(2^{\lceil n/2 \rceil})$ stored pairs; polynomial factors are suppressed in exponential-rate comparisons.

4 Proposed Algorithm

Our hybrid algorithm operates in three phases: classical LP preprocessing, quantum branch-and-bound over the reduced core, and classical post-processing.

4.1 Phase 1: LP Preprocessing via Reduced-Cost Variable Fixing

The classical preprocessing exploits the LP relaxation and reduced-cost analysis to provably partition items into three disjoint sets.

Every optimal assignment respects the decisions in F_1 and F_0. Let $m = |C|$ and $\alpha = m/n$.

4.2 Phase 2: Quantum Branch-and-Bound on Core

We define an implicit B&B tree over the m core items, with a fixed threshold and LP pruning. Detection and witness recovery are analyzed in Proposition 1; the explicit matrices used for component simulation are not a full gate-level walk implementation.

Algorithm 1 Reduced-Cost Variable Fixing

Require: Items $(w_i, v_i)_{i=1}^n$, capacity W
Ensure: Partition (F_1, C, F_0), residual capacity W'

Exclude items with $w_i > W$; return all remaining items if they fit
Compare efficiencies by exact rational arithmetic; sort decreasingly
Initialize $F_1, C, F_0 \leftarrow \emptyset$
Compute greedy lower bound Z_{LB}
Compute Dantzig upper bound Z_{LP}
for each item $i \in \{1, \ldots, n\}$ **do**
  if $x_i^{LP} = 1$ (item before split) **then**
    $Z_i' \leftarrow$ LP bound with item i forced out
    if $Z_i' < Z_{\mathrm{LB}}$ **then**
      $F_1 \leftarrow F_1 \cup \{i\}$ {Provably in optimal}
    else
      $C \leftarrow C \cup \{i\}$
    end if
  else
    $Z_i' \leftarrow$ LP bound with item i forced in
    if $Z_i' < Z_{\mathrm{LB}}$ **then**
      $F_0 \leftarrow F_0 \cup \{i\}$ {Provably out of optimal}
    else
      $C \leftarrow C \cup \{i\}$
    end if
  end if
end for
$W' \leftarrow W - \sum_{i \in F_1} w_i$; $V_1 \leftarrow \sum_{i \in F_1} v_i$
return (F_1, C, F_0), W', V_1

4.2.1 Classical B&B Tree Structure

At each node of the B&B tree, we branch on a core item c_i (include/exclude) and compute the LP relaxation bound on the remaining subproblem. For a detection call at threshold τ, a node is pruned when its LP bound is strictly below τ. The threshold is fixed throughout that call, so the tree does not depend on a changing incumbent.

Bounding oracle. The LP relaxation for the residual knapsack at each B&B node is computable in $O(m)$ time classically. The LP relaxation defines the bounding heuristic h required by the abstract query model.

4.2.2 Reversible LP-Bound Oracle (h)

For fixed depth d, write a node bound as $U = P + rv_s/w_s$, where P includes the already selected and fully packed integer values, and $0 \le r < w_s$. Efficiencies are ordered exactly. Precompute the problem constants $q_i = \lfloor 2^k v_i/w_i \rfloor$ and compute

$$A = 2^k P + rq_s, \qquad AW_{\max} + 2^k \le \tau\, 2^k W_{\max}. \tag{6}$$

Table 2 DERIVED gate-list resources on $w = (2, 3, 4, 5)$, $v = (3, 4, 5, 8)$, $W = 8$, $k = 5$, with a five-bit threshold register. The final row includes coherent depth dispatch. Workspace uncomputation is included.

Prefix depth d	Qubits	Gates	Gate depth
0	46	133	57
1	545	1747	257
2	404	1285	211
3	287	907	165
4	155	475	121
Coherent d	591	9228	1622

The inequality is the pruning predicate; Lemma 1a specifies sufficient precision. The truncated value $A/2^k$ is not itself treated as an upper bound.

Existing reversible arithmetic includes elementary networks [13], Fourier addition [14], and restoring division [15]. Our new construction uses X, CNOT, and Toffoli gates for binary additions, comparisons, multiplexing, and multiplication by the compiled constants. No division of quantum inputs is needed. It computes the prefix weight and value, scans the remaining items in efficiency order, packs full items while they fit, and uses the first fractional item once. Infeasible prefixes are rejected separately. Register widths cover the full weight/value sums and the scaled cross products. Copying the final flag and reversing the computation restores all workspace, giving

$$|x_{1:d}, \tau, b, 0\rangle \longmapsto |x_{1:d}, \tau, b \oplus [U(x_{1:d}) < \tau], 0\rangle.$$

The coherent wrapper computes each depth-specific flag into a private bit, conditionally flips the output using a binary depth register, and clears the private bit. It reuses the clean work pool between depths. Thus depth is handled coherently; the complete walk remains a separate implementation task.

SIMULATED checks on the 700-instance benchmark (seeds and sizes in Section 6) exhaust all prefix, threshold-register and target-bit basis states at depth $d = 3$, with clean initial workspace: 15,859,712 columns, 0 predicate mismatches against exact rational LP bounds, and 0 columns with uncleared workspace. Precision is $k = 13$ or 14, chosen per instance by $2^k > W_{\max}^2$. The X/CNOT/Toffoli circuit is a permutation, so sparse statevectors propagate basis labels and amplitudes exactly through every gate. This checks arbitrary superpositions in the tested input subspace by linearity. It is not a check of all depths on those 700 instances.

For $(w, v, W) = ((2, 3, 4, 5), (3, 4, 5, 8), 8)$, $((3, 5, 7, 8), (3, 5, 7, 8), 13)$, and $((4, 7, 6), (5, 6, 2), 7)$, every prefix depth and every threshold/target input was additionally checked: respectively 1984, 1984, and 480 columns at $k = 5, 7, 6$, 0 mismatches and 0 uncleared-workspace columns (SIMULATED). The coherent-depth wrappers on these same instances at $k = 5, 7, 6$ exhaust all prefix, binary-depth, threshold and target encodings: respectively 8192, 8192 and 1024 columns, 0 mismatches and 0

Algorithm 2 Threshold Search and Witness Recovery (Phase 2)

Require: Core C, residual capacity W', failure budget δ, certified tree-size bound B
Ensure: Core assignment x_C^*, except with probability at most δ

$S \leftarrow \sum_{i \in C} v_i$, $L \leftarrow \lceil \log_2(S+1) \rceil$, $M \leftarrow L + m$
Set each detection's failure probability to δ/M; solve $M = 0$ directly
$\ell \leftarrow 0$, $u \leftarrow S + 1$
while $u - \ell > 1$ **do**
    $\tau \leftarrow \lfloor (\ell + u)/2 \rfloor$
    Detect a feasible completion of value $\geq \tau$ in its fixed-threshold tree
    If detected, set $\ell \leftarrow \tau$; otherwise set $u \leftarrow \tau$
end while
$Z \leftarrow \ell$; initialize empty prefix with weight/value zero
for each item in the fixed efficiency order **do**
    Test whether extending the prefix by 0 permits a completion of value $\geq Z$
    Choose 0 if yes and 1 otherwise; update prefix weight and value
end for
Check the final assignment's capacity and value; report failure if invalid
return x_C^*

uncleared-workspace columns (SIMULATED). An independent dense Qiskit statevector check on $w = (2)$, $v = (1)$, $W = 2$, $d = 1$, $k = 3$ gave maximum amplitude error 5.56×10^{-17} (SIMULATED).

4.2.3 Quantum Walk Application

We apply Montanaro's quantum B&B framework [5] to the implicit tree defined by our LP-bounded branching strategy:

Recovery uses the prefix's accrued value and weight to form each residual threshold and capacity; empty or infeasible residual problems are decided classically. At most $L + m$ detections are made. The bound B must cover both value-search and recovery trees, not just the trees encountered during value search. Proposition 1 accounts for these calls and their error allocation.

4.2.4 Reversible Knapsack Oracle (Answer-Agnostic)

The separate phase predicate V marks feasible$(x) \wedge$ value$(x) \geq \tau$ using the problem data and a supplied threshold. The existing Draper-arithmetic construction accumulates weight and value, checks the two signed differences, flips the phase, and uncomputes both accumulators. On $w = (2, 3, 4, 5)$, $v = (3, 4, 5, 8)$, $W = 8$, dense statevector checks covered all 16 selection states at each of the 22 thresholds $\tau = 0, \ldots, 21$: 0 mismatches, with maximum amplitude error below 5.60×10^{-14} (SIMULATED). These are component checks, not executions of Algorithm 2 as a full quantum circuit.

4.3 Phase 3: Classical Post-Processing

Algorithm 3 Classical Reconstruction

Require: (F_1, C, F_0), quantum solution x^*_C, capacity W
Ensure: Globally optimal solution x^*
$x^* \leftarrow F_1 \cup \{i \in C : x^*_{C,i} = 1\}$
$Z^* \leftarrow \sum_{i \in x^*} v_i$
return x^*, Z^*

Since reduced-cost fixing is exact, the combined solution $F_1 \cup x^*_C$ is globally optimal. No swap verification is needed.

4.4 Overall Algorithm

Algorithm 4 LP-Pruned Quantum B&B Knapsack Solver

Require: Items $(w_i, v_i)_{i=1}^n$, capacity W, failure budget δ
Ensure: Optimal solution x^* and value Z^*, except with probability δ
Phase 1: $(F_1, C, F_0), W', V_1 \leftarrow$ RC-Fix(w, v, W) {Alg. 1}
$m \leftarrow |C|$; set certified $B \leftarrow 2^{m+1} - 1$ unless a smaller certificate is supplied
if $m = 0$ **then**
  return $x^* = F_1$, $Z^* = V_1$
else if a configured classical fallback is selected **then**
  Solve core classically via B&B
else
  Phase 2: $x^*_C \leftarrow$ Quantum-B&B(C, W', δ, B) {Alg. 2}
end if
Phase 3: $x^* \leftarrow F_1 \cup x^*_C$
return x^*, $Z^* = V_1 + v_C(x^*_C)$

4.5 Algorithmic Extensions

4.5.1 Multidimensional Knapsack Extension

The classical fixing rule also applies to the d-dimensional 0/1 knapsack problem. We generalize the LP fixing by using the minimum of d single-constraint Dantzig bounds. Sorting items by per-constraint efficiencies v_i/w_{ij} yields provably valid upper bounds for reduced-cost fixing (because each single-constraint Dantzig bound maximizes the objective over a superset of the full d-dimensional feasible region, making their minimum a valid relaxation). Table 3 summarizes the empirical behavior of α across 20 independent trials per configuration for strongly correlated instances ($n = 24$). For these multidimensional instances, weights are drawn independently as $w_{ij} \sim U(1, 100)$,

and values are tied to the average weight via $v_i = \lfloor \frac{1}{d} \sum_{j=1}^{d} w_{ij} \rfloor + 10$. The results show that strongly correlated multidimensional instances heavily resist classical variable fixing, giving $\alpha = 1.0$ in all tested configurations with $d \geq 2$, meaning zero variables are fixed by Phase 1. The consequences for quantum query complexity require a separate analysis of the resulting pruned trees.

Table 3 Core size ratio α vs. dimensionality d for strongly correlated instances ($n = 24$, 20 trials)

Dimensionality	$d = 1$	$d = 2$	$d = 3$	$d = 4$
Mean core ratio (95% CI)	0.96 ± 0.06	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00

5 Theoretical Analysis

5.1 Correctness

Lemma 1a (Safe fixed-point pruning). *For positive integer weights and values, retain the exact efficiency order. Let* $\widetilde{U} = P + r\lfloor 2^k v_s/w_s \rfloor / 2^k$ *approximate any feasible node or forced-item LP bound* $U = P + rv_s/w_s$*, with integer* P*,* $0 \leq r < w_s \leq W_{\max}$*. Then*

$$0 \leq U - \widetilde{U} < \varepsilon(k) = W_{\max} 2^{-k}.$$

If $\varepsilon(k) < 1/W_{\max}$*, the rule* $\widetilde{U} \leq \tau - 1/W_{\max}$ *is equivalent to* $U < \tau$ *for every integer threshold* τ*. This sufficient condition preserves both the fixing decisions at* $\tau = Z_{LB}$ *and the node-pruning decisions in Phase 2.*

Proof. Flooring gives $0 \leq v_s/w_s - \lfloor 2^k v_s/w_s \rfloor / 2^k < 2^{-k}$; multiplication by $r < W_{\max}$ gives the error bound. Since P and τ are integers, $U < \tau$ implies $U \leq \tau - 1/w_s \leq \tau - 1/W_{\max}$, so the truncated test prunes. If $U \geq \tau$, then $\widetilde{U} > U - \varepsilon(k) > \tau - 1/W_{\max}$, so it does not prune. If there is no fractional item, U is integral and the same argument applies with zero truncation error. Equation (6) implements the comparison by exact integer cross multiplication. □

The condition $2^k > W_{\max}^2$ is sufficient, not necessary for every instance: dyadic efficiencies may be represented exactly at lower precision. Pure truncation with the unadjusted test is unsafe even at equality. For example, excluding $(w, v) = (7, 6)$ from $w = (4, 7, 6)$, $v = (5, 6, 2)$, $W = 7$ gives $U = 5 + 3(2/6) = Z_{LB} = 6$, but $\widetilde{U} < 6$ at every finite precision. The robust rule avoids this false pruning when the lemma's precision condition holds. It guarantees decisions, not equality of approximate and rational bound values.

Theorem 1. *With exact fixing, valid node predicates, a certified tree-size bound for every detection call, and the error allocation in Algorithm 2, Algorithm 4 returns an optimal assignment with probability at least* $1 - \delta$*.*

Proof. If forcing the opposite value of a variable yields an LP bound below a feasible lower bound, no optimal assignment has that opposite value. Thus every optimum respects all fixed decisions. Conditional on correct detection answers, binary search

maintains an achievable lower threshold and an unachievable upper threshold and ends at the optimal core value. During recovery, the current prefix has a completion attaining that value. If the zero extension has no such completion, the one extension must have one; this preserves the invariant until a complete optimal assignment is obtained. Reconstruction adds the fixed-in items. There are at most $M = L + m$ adaptive calls, each with conditional failure probability at most δ/M; the union bound gives total failure at most δ. The final capacity/value check detects an invalid recovered witness but does not by itself certify optimality after erroneous detections. □

Lemma 1b (Dual bound for forced assignments). *For any item $i \neq s$, the exact forced-item LP bound satisfies $Z_i' \leq Z_{LP} - w_i|e_i - e_s|$. This upper bound does not require the split item to remain unchanged.*

Proof of Lemma 1b. Set $\lambda = e_s$. The LP dual bound is $Z_{LP} = \lambda W + \sum_j \max(0, v_j - \lambda w_j)$. Removing an item before the split deletes its nonnegative term; forcing an item after the split in changes the bound by $v_i - \lambda w_i \leq 0$. Optimizing the restricted LP cannot exceed either resulting dual bound, giving $Z_i' \leq Z_{LP} - w_i|e_i - e_s|$. This proof also covers ties with zero reduced cost. □

Theorem 2 (Core Containment). *Let C_{RC} be the reduced-cost core produced by Algorithm 1, and let C_{δ^*} be the δ-band core from Definition 3 with $\delta^* = \frac{Z_{LP} - Z_{LB}}{w_{\min} \cdot e_s}$, where $w_{\min} = \min_i w_i$. Assume a split item exists and items are sorted in nonincreasing efficiency order. Then $C_{RC} \subseteq C_{\delta^*}$.*

Proof. For $i = s$, containment is immediate. For any other $i \in C_{RC}$, item i was not fixed and $Z_i' \geq Z_{LB}$. By Lemma 1b, $Z_i' \leq \hat{Z}_i' = Z_{LP} - w_i|e_i - e_s|$. Thus, $Z_{LB} \leq Z_{LP} - w_i|e_i - e_s|$, which rearranges to $w_i|e_i - e_s| \leq Z_{LP} - Z_{LB}$, and therefore $|e_i - e_s| \leq \frac{Z_{LP} - Z_{LB}}{w_i} \leq \delta^* \cdot e_s$. This implies $i \in C_{\delta^*}$. □

Remark. The proof uses an upper bound, so a change of split item does not invalidate containment. It also gives the tighter item-dependent bound $|e_i - e_s| \leq (Z_{LP} - Z_{LB})/w_i$.

Proposition 2 (Composed Counting Bound). *Assume the hypotheses of Theorem 2, and that there exists a minimum efficiency spacing $\Delta > 0$ such that $|e_i - e_j| \geq \Delta$ for all $i \neq j$. Then the core size is bounded by $|C_{RC}| \leq \lfloor \frac{2(Z_{LP} - Z_{LB})}{w_{\min} \cdot \Delta} \rfloor + 1$.*

Proof. By Theorem 2, $|C_{RC}| \leq |C_{\delta^*}|$. The band C_{δ^*} has width $2\delta^* e_s$. With spacing Δ, this interval contains at most $\lfloor 2\delta^* e_s / \Delta \rfloor + 1$ items. □

Tree-count consequence. A full binary tree of depth m contains $2^{m+1} - 1$ vertices, so every threshold or recovery tree has at most this many nodes. If $K = \min\{n, \lfloor 2(Z_{LP} - Z_{LB})/(w_{\min}\Delta) \rfloor + 1\}$, Proposition 2 gives $m \leq K$ and hence $B = 2^{K+1} - 1$ is a certified bound for Proposition 1. This is a worst-case bound; positive spacing is not guaranteed in general, including tied-efficiency instances.

Remark (Gap alone does not bound the core). For $n \geq 2$, set $q = \lfloor n/2 \rfloor$, $M_0 = 2n^2$, $w_i = 2(M_0 + i)$, $v_i = 2$, and

$$W = \sum_{i=1}^{q} w_i + (M_0 + q + 1).$$

All data are positive integers and $e_i = 1/(M_0 + i)$ are distinct. Every q-item subset fits: the difference between the heaviest and lightest q-subsets is $2q(n - q) \leq n^2/2 <$

$M_0 + q + 1$. No $(q+1)$-item subset fits, because the lightest such subset adds $w_{q+1} = 2(M_0+q+1)$ to the lightest q-subset. Thus $Z^* = Z_{LB} = 2q$. The LP fills exactly half of item $q+1$, so $Z_{LP} - Z_{LB} = 1$ for every n. Each item is included in some optimum and excluded from another, so neither forced-item LP test can fix it: $|C_{RC}| = n$. Therefore no finite bound depending on the gap alone holds for this integer problem class. This does not rule out bounds using weights or other structure.

Proposition 1 (Detection and witness query complexity). *Let $S = \sum_{i \in C} v_i$, $L = \lceil \log_2(S+1) \rceil$, $m = |C|$, and $M = L + m > 0$. Suppose a supplied certified bound B covers every fixed-threshold tree queried during value search and prefix recovery. Under the assumptions of Montanaro's Theorem 1.1 [4], Algorithm 2 uses*

$$O\left(\sqrt{B \max(1,m)}\,(L+m)\log\frac{L+m}{\delta}\right) \tag{7}$$

queries, with failure probability at most δ.

Proof. Value search makes at most L detections and recovery at most m. Each call has depth at most m and error at most δ/M. Applying (5) and summing at most M calls gives (7); Theorem 1 gives correctness. □

The safe generic choice is $B = 2^{m+1} - 1$. A smaller bound requires a certificate; computing it by classical tree enumeration is not free. The simulator explicitly constructs its trees, so its running time is not the query bound. Classical preprocessing, arithmetic bit costs, coherent depth dispatch, and walk-reflection gate costs are also excluded. We make no general claim that this composition inherits Montanaro's adaptive optimal-tree bound [5].

5.2 Discussion: Structural Interpretation

The experiments suggest that the hybrid algorithm adapts across instance distributions. For uncorrelated uniform instances, classical B&B evaluates relatively few states. In the tested instances, subset-sum capacities generate much larger search spaces. The theorem-backed query bound indicates that, conditional on a large LP-pruned tree, square-root dependence on tree size can reduce query complexity relative to classical traversal of the same tree.

5.3 Comparison with Prior Work

Table 1 compares mechanisms. Equation (7) is a conditional oracle-query statement, not a measured runtime or a comparison with the best classical knapsack solver. In particular, an easy first-solution classical search need not traverse the entire threshold tree [4].

6 Experimental Results

6.1 Experimental Setup

The revised experiments use exact rational efficiency comparisons and LP bounds, with dynamic programming as the classical reference on the small-coefficient benchmark.

The value-search simulator now returns a prefix-recovered assignment and includes its detection calls in the scheduled total. It computes analytical acceptance/majority probabilities rather than sampling a physical quantum execution. The empirical phase-estimation constant and fixed repetition count in the simulation are not a universal certification of the error assumptions in Theorem 1.

Numbers are classified as follows: classical solve outcomes, node counts and timings are `MEASURED`; exact bound calculations, fitted statistics and gate-list resource counts are `DERIVED`; quantum-operator and statevector outputs are `SIMULATED`; chosen quantum query budgets are `SCHEDULED`; manually chosen sizes, cutoffs and precision policies are `HARDCODED` configurations. No scheduled budget is interpreted as a measured quantum cost.

The classical and node-predicate benchmark has five Pisinger-style classes [8]: uncorrelated, weakly correlated, strongly correlated, subset sum, and inverse strongly correlated. For each class, $n \in \{12, 14, 16, 18, 20, 22, 24\}$ and seeds $1000n + t$, $t = 0, \ldots, 19$, give 700 instances. The coefficient range parameter is $R = 100$, with capacity $\lfloor \sum_i w_i/2 \rfloor$; the generators and exact inputs are reproducible from these specifications. Exact fixing plus core solving gave 0 optimum mismatches against dynamic programming on all 700 instances, with 0 node-capped cases (MEASURED). This establishes agreement on this tested set, not a universal empirical proof.

The analytical-majority optimization simulator was separately checked on 135 configurations: five generators, $n = 0, \ldots, 8$, three seeds $1000n + t$ per size, $R = 15$, phase-estimation constant 2, $K = 11$, and a 2000-node cap. It returned 0 value/witness mismatches and 0 capped cases (SIMULATED). These small tests do not establish the asymptotic query bound experimentally.

6.2 Variable Fixing Results

Table 4 shows the filtering ratio α across instance types.

Table 4 MEASURED mean fixing ratios at $n = 24$, 20 seeds per class, from the exact-arithmetic rerun.

Instance Type	$\boldsymbol{\alpha}$	$\lvert F_1\rvert/n$	$\lvert F_0\rvert/n$	**LP Gap**
Uncorrelated	0.252	0.483	0.265	Positive
Weakly correlated	0.748	0.140	0.112	Mixed
Strongly correlated	0.958	0.035	0.006	Positive
Subset-sum	1.000	0.000	0.000	Zero in this set
Inverse strongly	0.735	0.062	0.202	Positive

6.3 Spectral Checks of the Quantum Walk

The spectral audit uses $R = 50$, $n \in \{8, 10, 12, 14, 16, 18\}$, and seeds $1000n + t$, $t = 0, \ldots, 19$, for each of the five classes. Each seed has a marked threshold equal to the exact residual optimum and an unmarked threshold one greater: 1200 scheduled

Table 5 SIMULATED spectral audit status counts. The 1200 conditions include bypassed and capped cases.

Category	Count
Completed marked	489
Completed unmarked	400
Marked tree cap	74
Immediately pruned unmarked root	163
Empty-core seed conditions	74
Total	1200

conditions. These answer-known thresholds are used only to audit detection, not to run optimization.

Of these conditions, 889 trees completed with $T \leq 2000$: 489 marked and 400 unmarked. The remaining conditions are 74 marked tree caps, 74 conditions bypassed at 37 empty-core seeds, and 163 immediately pruned unmarked roots (SIMULATED status counts). None is silently discarded. Median core size is 16 for capped cases and 10 for completed cases (DERIVED), limiting generalization to larger trees.

The fresh audit uses orthogonal real Schur blocks so degenerate eigenspaces do not require treating nonorthogonal eigenvectors as a probability basis. On the 889 completed trees, spectral and direct sparse-walk acceptance calculations differ by at most 9.49×10^{-12} (SIMULATED). We check the effective spectral-gap inequality $\mu(\Phi) \leq \Phi^2 Tm/4$ for unmarked trees at $\Phi = c/\sqrt{Tm}$, $c \in \{1/8, 1/4, 1/2, 1, 2\}$, and phase-zero weight at least $1/2$ for marked trees, with numerical tolerances. These are the conditions in the proof of Montanaro's Lemma 2.4 and effective spectral-gap Lemma 2.1 [4], with eigenphase $\Phi = 2\theta$.

The results are 400/400 unmarked trees satisfying all tested inequalities (0 violations), 489/489 marked trees satisfying the phase-zero test (0 violations), and 889/889 analytical-majority decisions agreeing with marked status (0 mismatches), all SIMULATED. The phase-estimation constant is 2 and $K = 11$. These are component checks on the completed subset, not measurements of optimization query scaling, minimum spectral gaps, or a speedup.

6.4 Classical Runtime and Memory Characterization

Archived instrumentation used `time.perf_counter_ns()` and `tracemalloc`. Its 1500 method records correspond to 500 matched instance configurations at $n \in \{12, 14, 16, 18, 20\}$, with three methods per configuration. The classical baseline already uses LP bounds; the comparison measures the additional cost of fixing. Across the five classes, median paired baseline/fixing runtime ratios range from 0.267 to 0.533, and median fixing-solver peak allocations range from 1384 to 1416 bytes (DERIVED from archived MEASURED records). These timings belong to the archived implementation and were not remeasured for the revised exact arithmetic. Simulation runtime is not physical quantum execution time; no wall-clock quantum speedup is reported.

Table 6 Archived MEASURED classical timings: both solvers reported certified optimality. Times shown as 0.0000 s are rounded, not zero.

Instance Class	n	Successes	Core m	OR-Tools (s)	Our B&B (s)
Uncorrelated	100	5/5	18.2	0.0001	0.0031
Uncorrelated	200	5/5	27.2	0.0000	0.0210
Uncorrelated	500	5/5	78.0	0.0002	0.0984
Uncorrelated	1000	5/5	75.6	0.0001	0.3301
Weakly correlated	100	5/5	79.4	0.0000	0.0032
Weakly correlated	200	5/5	56.4	0.0002	0.0107
Weakly correlated	500	5/5	91.6	0.0001	0.0729
Weakly correlated	1000	5/5	126.2	0.0001	0.3253
Subset sum	100	5/5	100.0	0.0000	0.0028
Subset sum	200	5/5	200.0	0.0000	0.0109
Subset sum	500	5/5	500.0	0.0000	0.0733
Subset sum	1000	5/5	1000.0	0.0001	0.2970
Strongly correlated	100	3/5	100.0	0.0809	0.0877
Strongly correlated	200	2/5	200.0	0.0130	0.0607
Inverse strongly	100	3/5	36.0	0.0563	0.0191
Inverse strongly	200	1/5	25.0	0.0017	0.0108

6.5 Comparison Against Google OR-Tools at Scale

The archived classical benchmark compared LP fixing plus classical B&B against Google OR-Tools across 5 instance classes, with 5 trials each, scaling n up to 1000 for tractable classes and hard-stopping at $n = 500$ for strongly and inverse-strongly correlated classes once intractability was reached. Across every class and size, Phase 1 (reduced-cost fixing) resolved 0 of the tested instances outright; Phase 2 core-solving was required in 100% of cases tested, providing large-scale context for the variable fixing effectiveness reported for smaller n in Table 4. Of 90 total instances attempted across all classes and sizes, 69 were fully solved to certified optimality by both solvers, with zero mismatches between this paper's reported optimal value and OR-Tools' optimal value in every one of those 69 cases; these are value-agreement checks for those completed classical runs.

For the remaining 21 instances, cutoffs were reached (OR-Tools at 20s, our B&B at 1,000,000 nodes); because cutoff time is not solve time, no timing comparison is drawn from these rows, and Table 7 reports only the timeout counts. Table 6 details the performance for the 69 fully solved instances.

Table 7 Archived MEASURED cutoff counts; these rows are excluded from solve-time comparisons.

Instance Class	n	OR-Tools Timeouts	Our B&B Timeouts
Strongly correlated	100	2/5	2/5
Strongly correlated	200	3/5	3/5
Strongly correlated	500	5/5	5/5
Inverse strongly	100	2/5	2/5
Inverse strongly	200	4/5	4/5
Inverse strongly	500	5/5	5/5

Table 8 DERIVED exploratory fits for actual classical core-tree node counts; positive-gap cases only.

Class	Included	R^2	CV mean R^2
Uncorrelated	140	0.872	0.861
Weakly correlated	139	0.802	0.775
Strongly correlated	140	0.701	0.617
Inverse strongly	140	0.770	0.688

6.6 Large-Coefficient Benchmark

An additional MEASURED benchmark uses $R = 10^6$, $n \in \{20, 24, 28\}$, three trials per class, and seeds $20261022 + 100n + t$, $t = 0, 1, 2$: 45 instances. Meet-in-the-middle supplies an independent exact reference. OR-Tools 9.15.6755 solved all 45 within its one-second limit, with 0 optimum mismatches. Exact LP fixing plus classical B&B completed 36 with 0 mismatches; the other 9 reached the 100,000-node cap and remain included as capped cases. Dynamic programming was not run on these large-coefficient instances. This small benchmark does not establish a quantum advantage or replace a comparison with a specialized solver such as COMBO [9].

6.7 Exploratory Tree-Size Regression

We fit $\log N_{\text{core}} \sim \log(Z_{LP} - Z^*) + m$ to the exact rerun, where N_{core} is the actual node count of classical incumbent-based core solving. It is not the maximum fixed-threshold tree size. The response is counted by the solver, not computed from the predictors. Zero gaps are ineligible for this logarithmic model: 141/700 cases (all 140 subset-sum cases and one weakly correlated case) are explicitly excluded; no cases were capped. On the remaining 559 cases, pooled $R^2 = 0.783$ and five-fold grouped cross-validation mean $R^2 = 0.776$ (DERIVED). Groups keep equal (n, seed) configurations across classes in one fold.

These fits describe the tested $n \leq 24$, $R = 100$ distributions and provide neither a universal tree-size bound nor independent evidence for quantum query scaling. Regressing a query budget selected from tree size on that same tree size would be circular, so no such validation is used.

7 Discussion

7.1 Instance-Dependent Behavior

The exact rerun shows different contributions from classical fixing and residual search. At $n = 24$, the uncorrelated mean core contains 6.05 items and its classical core solve visits 24.5 nodes, whereas strongly correlated means are 23 items and 3076.7 nodes (MEASURED). Subset-sum fixing leaves every item in this tested set, yet its mean classical core solve visits only 126.15 nodes: failure to fix variables alone does not imply a hard residual search. Classical incumbent-based node counts and fixed-threshold walk tree sizes are different metrics.

The query theorem gives square-root dependence on a certified size bound, with explicit depth, recovery and error costs. It does not ensure a reduction against the fastest classical solver on each distribution. Nor does a small core ratio establish a constant-size core as n grows.

7.2 Limitations

Implementation scope. The reversible circuits implement tested node/threshold predicates, including coherent depth dispatch on the specified small instances. Complete walk reflections and a full optimization circuit have not been composed or executed. Sparse permutation statevectors for the new Boolean circuits and explicit tree-space walk simulations test different objects. The supplied bound B must be justified for every detection tree; classical enumeration to obtain it cannot be hidden as free preprocessing in a runtime claim.

Hardware evidence. No hardware execution of the revised algorithm is reported. Archived IBM count summaries lack retained job identifiers and circuit hashes sufficient to establish that the current answer-agnostic circuit was executed; they are not used as evidence for the present algorithm or a universal hardware size boundary.

Resource estimates. Gate-list counts are component resources, not a full fault-tolerant resource estimate. A controlled-phase gate cannot simply be counted as a Toffoli, and an asymptotic or heuristic rotation-synthesis estimate is not a measured T-count or a certified lower bound. We therefore do not infer a numerical crossover by equating T gates with classical nodes.

Classical baselines and scope. Capacity-based dynamic programming requires $O(nW)$ operations and can use $O(W)$ working storage. It is a strong baseline when coefficients are small. The large-coefficient checks are still small in item count, and COMBO has not been benchmarked here. No general advantage over specialized classical solvers is claimed.

Detection reliability and selection. The simulated phase-estimation constant is calibrated for the reported tests, not proved universal. The 889 completed spectral trees exclude the explicitly counted capped, trivial and bypassed conditions. Their agreement supports those implemented tests on that subset; it does not establish an asymptotic optimization complexity experimentally.

8 Conclusion

We studied exact LP fixing followed by fixed-threshold quantum tree detection for integer knapsack. The revised algorithm explicitly recovers a witness and allocates error across value-search and recovery calls. Its query bound is conditional on a certified bound for all queried trees. The core containment argument identifies the role of efficiency spacing, and the integer counterexample has an unbounded core at the same gap of one.

The finite-precision lemma certifies pruning decisions, including equality cases. The new node circuits had 0 mismatches and 0 uncleared-workspace columns on 15,859,712 sparse statevector columns covering depth-three prefix/threshold/target inputs for the specified 700-instance benchmark at $k \in \{13, 14\}$ (SIMULATED). The exact classical

rerun had 0 optimum mismatches on all 700 cases (MEASURED). The fresh 889-tree completed spectral subset had 0 tested endpoint violations and 0 analytical detection disagreements (SIMULATED). These scoped component results establish neither a complete gate-level quantum optimization algorithm nor a wall-clock quantum speedup.

Future work includes coherent composition of the predicate circuits with the walk, tighter certified tree-size bounds for structured distributions, and comparisons with specialized classical solvers on larger, hard large-coefficient instances. Any practical quantum advantage requires separate evidence.

9 Data Availability and Reproducibility

The project repository is https://github.com/kumudinimetta-lab/Knapsack_Quantum. The revision's scripts and separately generated results are supplied with this manuscript source under `simulation/` and `research_log/`; historical raw records are preserved. The revision audit specifies generator parameters, seeds, cutoffs, predicate mismatch counts, workspace checks, and source hashes. New files must be included in the deposited revision; their presence in the public repository is not assumed until that version is published. The reproduction guide distinguishes classical reference solves, sparse/dense statevectors, analytical walk simulation, scheduled query budgets, and archived measurements.

Statements and Declarations

Funding, competing interests, and author contributions. These author-supplied declarations require confirmation before submission; no funding status, absence of conflicts, or individual contribution is inferred here.

Use of AI tools. An OpenAI coding assistant assisted with source checking, mathematical revision, code changes, and manuscript editing. The authors must independently review the resulting analysis, code, and text and take responsibility for the submitted version.

Acknowledgment

The authors thank the Department of Computer Science and Engineering, VNR Vignana Jyothi Institute of Engineering and Technology, Hyderabad, for institutional support. We also wish to extend our sincere gratitude to Dr. C. Thirumal, Professor at VNR Vignana Jyothi Institute of Engineering and Technology, for his valuable guidance and support. The acknowledgment of institutional support does not substitute for the funding declaration required before submission.

References

[1] Horowitz, E., Sahni, S.: Computing partitions with applications to the knapsack problem. Journal of the ACM **21**(2), 277–292 (1974) https://doi.org/10.1145/321812.321823

[2] Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proc. 28th Annual ACM Symposium on Theory of Computing (STOC), pp. 212–219 (1996). https://doi.org/10.1145/237814.237866

[3] Brassard, G., Høyer, P., Mosca, M., Tapp, A.: Quantum amplitude amplification and estimation. Contemporary Mathematics **305**, 53–74 (2002)

[4] Montanaro, A.: Quantum-walk speedup of backtracking algorithms. Theory of Computing **14**(15), 1–24 (2018) https://doi.org/10.4086/toc.2018.v014a015

[5] Montanaro, A.: Quantum speedup of branch-and-bound algorithms. Physical Review Research **2**(1), 013056 (2020) https://doi.org/10.1103/PhysRevResearch.2.013056

[6] Dantzig, G.B.: Discrete-variable extremum problems. Operations Research **5**(2), 266–277 (1957) https://doi.org/10.1287/opre.5.2.266

[7] Kellerer, H., Pferschy, U., Pisinger, D.: Knapsack Problems. Springer, Berlin, Germany (2004). https://doi.org/10.1007/978-3-540-24777-7

[8] Pisinger, D.: Core problems in knapsack algorithms. Operations Research **47**(4), 570–575 (1999) https://doi.org/10.1287/opre.47.4.570

[9] Martello, S., Pisinger, D., Toth, P.: Dynamic programming and strong bounds for the 0-1 knapsack problem. Management Science **45**(3), 414–424 (1999) https://doi.org/10.1287/mnsc.45.3.414

[10] Wilkening, S., Lefterovici, A.-I., Binkowski, L., Perk, M., Fekete, S.P., Osborne, T.J.: A quantum algorithm for solving 0-1 knapsack problems. npj Quantum Information **11**, 146 (2025) https://doi.org/10.1038/s41534-025-01097-8

[11] Creemers, S., Armas, L.F.P.: Discrete optimization: A quantum revolution? European Journal of Operational Research **323**(2), 378–408 (2025) https://doi.org/10.1016/j.ejor.2024.12.016

[12] Bożejko, W., Burduk, A., Pempera, J., Uchroński, M., Wodecki, M.: Optimal solving of a binary knapsack problem on a D-Wave quantum machine and its implementation in production systems. Annals of Operations Research **364**, 2111–2126 (2026) https://doi.org/10.1007/s10479-024-06025-1 . First published online 13 May 2024

[13] Vedral, V., Barenco, A., Ekert, A.: Quantum networks for elementary arithmetic operations. Physical Review A **54**(1), 147–153 (1996) https://doi.org/10.1103/PhysRevA.54.147

[14] Draper, T.G.: Addition on a Quantum Computer (2000). https://arxiv.org/abs/quant-ph/0008033

[15] Thapliyal, H., Varun, T.S.S., Munoz-Coreas, E.: Quantum Circuit Design of Integer Division Optimizing Ancillary Qubits and T-Count (2016). https://arxiv.org/abs/1609.01241